

Guia: Adicionar uma Nova Entidade

Fluxo de alteração camada a camada (DDD)

Passo a passo prático para incluir uma nova entidade/recurso no projeto sem quebrar a arquitetura. A ordem segue a regra de dependência do DDD: começa-se pelo domínio (interno) e avança-se até a API (externo). Ao final há um exemplo completo (entidade Cupom) e uma lista de armadilhas comuns.

1. Princípio que guia toda alteração

A dependência aponta para dentro: **API** → **Application** → **Domain** ← **Infrastructure**. Portanto, defina primeiro o que a entidade **é** e o que ela **faz** (domínio), depois **como** ela é orquestrada (aplicação) e **onde** é persistida (infraestrutura), e só então **como** é exposta (API). Nunca importe SQLAlchemy ou FastAPI dentro de domain/.

2. Checklist (ordem recomendada)

#	Camada / arquivo	O que fazer
1	domain/enums.py	Enums novos (se houver estados/tipos)
2	domain/entities/<nome>.py	Entidade com atributos + comportamento (invariantes)
3	core/exceptions.py	Exceções de negócio da entidade (com status_code)
4	domain/repositories/<nome>_repository.py	Interface (port): métodos abstratos
5	infrastructure/models.py	Modelo ORM (tabela)
6	infrastructure/mappers.py	Conversão ORM ■ entidade
7	infrastructure/repositories/<nome>_repository_impl.py	Implementação SQLAlchemy do port
8	application/services/<nome>_service.py	Caso de uso (orquestra domínio + repos)
9	schemas/<nome>.py	DTOs Pydantic (request/response, alias camelCase)
10	api/presenters.py	Função entidade -> schema de resposta
11	api/<nome>s.py	Router FastAPI (rotas, auth/roles)
12	core/deps.py	Fábrica get_<nome>_service + wiring dos repos
13	main.py	include_router do novo recurso
14	infrastructure/seed.py	Dados iniciais (se necessário)
15	tests/ + Postman	Cenários positivos e negativos; recriar o banco

3. Exemplo completo: entidade Cupom

Objetivo: cadastrar cupons de desconto (código + percentual) e validar um cupom. Mostra cada camada em um recorte mínimo e funcional.

Passo 2 — Entidade de domínio (regras dentro da entidade)

```
# app/domain/entities/cupom.py
from dataclasses import dataclass
from app.core.exceptions import CupomInvalidoError

@dataclass
class Cupom:
    id: int | None
    codigo: str
    percentual: int          # 1..100
    ativo: bool = True

    def __post_init__(self):
        if not (1 <= self.percentual <= 100):
            raise CupomInvalidoError(message="Percentual deve estar entre 1 e 100.")

    def aplicar(self, valor_cents: int) -> int:
        if not self.ativo:
            raise CupomInvalidoError(message="Cupom inativo.")
        return valor_cents - (valor_cents * self.percentual // 100)
```

Passo 3 — Exceção de negócio

```
# app/core/exceptions.py (acrescentar)
class CupomInvalidoError(DomainError):
    error = "CUPOM_INVALIDO"
    message = "Cupom inválido."
    status_code = 422
```

Passo 4 — Porta (interface)

```
# app/domain/repositories/cupom_repository.py
from abc import ABC, abstractmethod
from app.domain.entities.cupom import Cupom

class CupomRepository(ABC):
    @abstractmethod
    def add(self, c: Cupom) -> Cupom: ...
    @abstractmethod
    def get_by_codigo(self, codigo: str) -> Cupom | None: ...
```

Passos 5 e 6 — ORM e mapper

```
# app/infrastructure/models.py (acrescentar)
class CupomORM(Base):
    __tablename__ = "cupons"
    id = Column(Integer, primary_key=True)
    codigo = Column(String, unique=True, nullable=False)
    percentual = Column(Integer, nullable=False)
    ativo = Column(Boolean, nullable=False, default=True)

# app/infrastructure/mappers.py (acrescentar)
def cupom_to_domain(o):
    return Cupom(id=o.id, codigo=o.codigo, percentual=o.percentual, ativo=o.ativo)
```

Passo 7 — Repositório SQLAlchemy

```
# app/infrastructure/repositories/cupom_repository_impl.py
from sqlalchemy.orm import Session
from app.domain.entities.cupom import Cupom
from app.domain.repositories.cupom_repository import CupomRepository
from app.infrastructure import mappers, models

class SQLCupomRepository(CupomRepository):
    def __init__(self, db: Session): self.db = db
    def add(self, c: Cupom) -> Cupom:
        orm = models.CupomORM(codigo=c.codigo, percentual=c.percentual, ativo=c.ativo)
        self.db.add(orm); self.db.commit(); self.db.refresh(orm)
        return mappers.cupom_to_domain(orm)
    def get_by_codigo(self, codigo: str) -> Cupom | None:
        o = self.db.query(models.CupomORM).filter(models.CupomORM.codigo==codigo).first()
        return mappers.cupom_to_domain(o) if o else None
```

Passo 8 — Service (caso de uso)

```
# app/application/services/cupom_service.py
from app.core.exceptions import CupomInvalidoError
from app.domain.entities.cupom import Cupom
from app.domain.repositories.cupom_repository import CupomRepository

class CupomService:
    def __init__(self, cupons: CupomRepository): self.cupons = cupons
    def criar(self, codigo: str, percentual: int) -> Cupom:
        return self.cupons.add(Cupom(id=None, codigo=codigo, percentual=percentual))
    def validar(self, codigo: str) -> Cupom:
        c = self.cupons.get_by_codigo(codigo)
        if not c or not c.ativo:
            raise CupomInvalidoError()
        return c
```

Passos 9 a 11 — Schemas, presenter e router

```
# app/schemas/cupom.py
from pydantic import BaseModel, Field
class CupomCreate(BaseModel):
    codigo: str = Field(..., min_length=3)
    percentual: int = Field(..., ge=1, le=100)
class CupomResponse(BaseModel):
    id: int; codigo: str; percentual: int; ativo: bool

# app/api/presenters.py (acrescentar)
def cupom_out(c):
    return CupomResponse(id=c.id, codigo=c.codigo, percentual=c.percentual, ativo=c.ativo)

# app/api/cupons.py
from fastapi import APIRouter, Depends, status
from app.api.presenters import cupom_out
from app.core.deps import get_cupom_service, require_roles
from app.domain.enums import Role
from app.schemas.cupom import CupomCreate, CupomResponse
router = APIRouter(prefix="/cupons", tags=["Cupons"])

@router.post("", response_model=CupomResponse, status_code=status.HTTP_201_CREATED)
def criar(payload: CupomCreate,
          _u=Depends(require_roles(Role.ADMIN.value, Role.GERENTE.value)),
          service=Depends(get_cupom_service)):
    return cupom_out(service.criar(payload.codigo, payload.percentual))
```

Passos 12 e 13 — Wiring e registro

```
# app/core/deps.py (acrescentar)
from app.application.services.cupom_service import CupomService
from app.infrastructure.repositories.cupom_repository_impl import SQLCupomRepository
def get_cupom_service(db: DbDep) -> CupomService:
    return CupomService(SQLCupomRepository(db))

# app/main.py (acrescentar)
from app.api import cupons
app.include_router(cupons.router)
```

Passo 15 — Teste

```
# tests/test_cupons.py
def test_criar_cupom(client, admin_headers):
    r = client.post("/cupons", headers=admin_headers,
                    json={"codigo": "NORDESTE10", "percentual": 10})
    assert r.status_code == 201 and r.json()["percentual"] == 10
```

4. Armadilhas comuns

- Esquecer o **mapper** ou o **wiring** em `deps.py` — o erro só aparece em runtime. Rode `python -m compileall app` e os testes.
- Importar `SQLAlchemy/FastAPI` dentro de `domain/` — quebra a regra de dependência.
- Banco antigo sem a nova tabela: como usamos `create_all`, apague o arquivo `*.db` (ou rode os testes, que usam banco isolado) para recriar o schema. Em produção, isso seria uma migration (Alembic).
- Alias camelCase: lembre `ConfigDict(populate_by_name=True)` no schema de request e `serialization_alias` na resposta.
- Relacionamentos: defina FKs no ORM e, se precisar do nome de outra entidade na resposta, traga via `relationship` e preencha no mapper.

- Sempre adicione ao menos 1 cenário **negativo** (erro) no teste e no Postman.